

ui.space 全面详解

一、核心概述

`ui.space` 是 NiceGUI 框架中的弹性空间填充组件，基于 Quasar 的 `QSpace` 组件实现，核心作用是在弹性布局 (flexbox) 容器中自动填充剩余可用空间，实现元素的灵活对齐与布局分隔。其本质是利用 CSS Flexbox 的 `flex: 1` 特性，无需手动计算空间尺寸，即可快速实现“两端对齐”“垂直分布”等常见布局需求，是提升界面布局灵活性和响应式能力的关键组件。

二、基础用法

1. 水平布局两端对齐（核心场景）

在 `ui.row()`（水平弹性布局容器）中插入 `ui.space()`，可自动填充左右元素之间的剩余空间，实现两端对齐效果：

```
from nicegui import ui

# 水平布局容器，设置全宽和边框以便观察
with ui.row().classes('w-full border p-4'):
    ui.label('左侧元素')
    ui.space() # 填充左右元素间的剩余水平空间
    ui.label('右侧元素')

ui.run()
```

运行后效果：“左侧元素”居左，“右侧元素”居右，中间由 `ui.space` 自动填充空白空间，且空间会随容器宽度变化自适应调整。

2. 垂直布局上下分布

在 `ui.column()`（垂直弹性布局容器）中使用 `ui.space()`，可填充上下元素间的剩余垂直空间，实现上下分布效果：

```
from nicegui import ui

# 垂直布局容器，设置固定高度、边框以便观察
with ui.column().classes('h-32 border p-4 w-48'):
    ui.label('顶部元素')
    ui.space() # 填充上下元素间的剩余垂直空间
    ui.label('底部元素')

ui.run()
```

运行后效果：“顶部元素”居上，“底部元素”居下，中间空白区域由 `ui.space` 自动填充，适配容器高度变化。

3. 核心特性

- 布局依赖：仅在弹性布局容器（`ui.row()`、`ui.column()` 或自定义 `display: flex` 容器）中生效；
- 自适应能力：自动填充容器剩余空间，随容器尺寸变化实时调整；
- 无视觉样式：默认仅填充空间，无背景、边框等可见样式（可通过属性自定义）；
- 轻量高效：无需额外配置，一行代码即可实现复杂布局对齐。

三、属性 (Properties)

`ui.space` 继承了 NiceGUI 基础元素的核心属性，支持样式定制、状态管理、DOM 定位等扩展能力，关键属性如下：

属性名	类型	说明	新增版本	
<code>classes</code>	<code>Classes[Self]</code>	为组件添加 HTML 类（支持 Tailwind、Quasar 类），用于批量定制样式（如背景色、内边距）	-	
<code>client</code>	<code>Client</code>	绑定该元素所属的客户端实例，用于多客户端协作场景	-	
<code>html_id</code>	<code>str</code>	设定元素在 HTML DOM 中的唯一 ID，便于通过 JavaScript 操作或 CSS 定位	2.16.0	
<code>is_deleted</code>	<code>bool</code>	只读属性，标识元素是否已被删除	-	
<code>is_ignoring_events</code>	<code>bool</code>	只读属性，标识元素是否正在忽略事件（如点击、悬浮）	-	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	可设置属性，指定元素所属的父插槽（用于复杂组件嵌套场景）	-
<code>props</code>	<code>Props[Self]</code>	配置 Quasar 组件原生属性，扩展组件功能（如自定义弹性属性）	-	
<code>style</code>	<code>Style[Self]</code>	直接设置内联 CSS 样式，优先级高于 <code>classes</code> （如强制设置宽度、高度）	-	
<code>visible</code>	<code>BindableProperty</code>	绑定元素可见性（支持双向绑定），默认值为 <code>True</code> （显示）	-	

常用属性示例

(1) 通过 `classes` 定制填充空间样式

为 `ui.space` 添加背景和内边距，使其成为“可见的填充区域”：

```
from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    # 添加 Tailwind 类: 浅蓝色背景、16px 内边距、圆角
    ui.space().classes('bg-blue-100 p-4 rounded-lg')
    ui.label('右侧')

ui.run()
```

(2) 通过 `style` 强制限制填充空间尺寸

手动指定 `ui.space` 的最小宽度 / 高度，限制其填充范围：

```
from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    # 强制最小宽度 100px，即使容器剩余空间更小
    ui.space().style('min-width: 100px; background-color: #f0f0f0;')
    ui.label('右侧')

ui.run()
```

(3) 通过 `props` 扩展 Quasar 原生属性

利用 Quasar 弹性布局属性，自定义 `ui.space` 的弹性行为：

```
from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('元素1')
    # flex-grow: 2 表示该空间填充权重为 2，占据更多剩余空间
    ui.space(props='flex-grow=2 bg-green-100')
    ui.label('元素2')
    # flex-grow: 1 表示该空间填充权重为 1
    ui.space(props='flex-grow=1 bg-yellow-100')
    ui.label('元素3')

ui.run()
```

效果：元素 1 与元素 2 之间的空间宽度是元素 2 与元素 3 之间的 2 倍，实现加权填充。

四、核心方法 (Methods)

`ui.space` 提供了丰富的方法用于动态控制元素状态、样式、事件等，以下是高频使用的方法分类详解：

1. 可见性控制

(1) `set_visibility(visible: bool)`

直接设置 `ui.space` 是否可见（隐藏后不再填充空间）：

```
from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    space = ui.space().classes('bg-blue-100')
    ui.label('右侧')

# 2 秒后隐藏空间，元素会自动靠拢
ui.timer(2, lambda: space.set_visibility(False))

ui.run()
```

(2) 双向绑定可见性: `bind_visibility()`

将 `ui.space` 的可见性与目标对象的属性绑定，实现联动控制：

```
from nicegui import ui

class LayoutState:
    def __init__(self):
        self.show_space = True # 控制空间是否显示

state = LayoutState()

# 勾选框与空间可见性双向绑定
ui.checkbox('显示填充空间', value=state.show_space,
           on_change=lambda e: setattr(state, 'show_space', e.value))

with ui.row().classes('w-full border p-4 mt-2'):
    ui.label('左侧')
    ui.space().bind_visibility(state, 'show_space').classes('bg-blue-100')
    ui.label('右侧')

ui.run()
```

效果：勾选框勾选时显示空间（元素两端对齐），取消勾选时隐藏空间（元素靠拢）。

(3) 单向绑定: `bind_visibility_from()`/`bind_visibility_to()`

- `bind_visibility_from(target_object, target_name)`: 仅从目标对象同步可见性 (目标→组件) ;
- `bind_visibility_to(target_object, target_name)`: 仅从组件同步可见性 (组件→目标) 。

2. 样式与属性动态修改

(1) `default_classes()`/`default_props()`/`default_style()`

批量修改所有 `ui.space` 实例的默认样式 (需在实例化前调用) :

```
from nicegui import ui

# 全局配置所有 ui.space 的默认样式
ui.space.default_classes(add='bg-gray-100 p-2') # 默认灰色背景、2px 内边距
ui.space.default_style(add='min-height: 20px;') # 默认最小高度 20px
ui.space.default_props(add='flex-shrink=0') # 禁止空间收缩

# 所有实例都会继承上述默认样式
with ui.column().classes('h-40 border p-4'):
    ui.label('顶部')
    ui.space() # 继承默认样式
    ui.label('中间')
    ui.space() # 继承默认样式
    ui.label('底部')

ui.run()
```

(2) `update()`

手动触发组件在客户端的更新 (修改属性后需调用以生效) :

```
from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    space = ui.space()
    ui.label('右侧')

# 动态修改样式后, 调用 update() 同步到前端
space.classes('bg-purple-100')
space.update()

ui.run()
```

3. 事件与交互

(1) `on(type: str, handler: Callable)`

为 `ui.space` 绑定事件（如点击、鼠标悬浮），实现交互逻辑：

```
from nicegui import ui

def on_space_click(e):
    ui.notify('填充空间被点击了!')

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    # 绑定点击事件
    ui.space().classes('bg-green-100 cursor-pointer').on('click', on_space_click)
    ui.label('右侧')

ui.run()
```

支持的事件类型：`click`、`mouseover`、`mousedown` 等原生 DOM 事件，也可通过 `js_handler` 配置客户端 JavaScript 逻辑：

```
ui.space(js_handler='(e) => alert("客户端：空间被点击")').classes('bg-yellow-100')
```

(2) `tooltip(text: str)`

为 `ui.space` 添加悬浮提示：

```
from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    ui.space().tooltip('这是填充空间').classes('bg-blue-100')
    ui.label('右侧')

ui.run()
```

4. 元素管理

(1) `delete()`

删除 `ui.space` 组件（不可逆，删除后不再填充空间）：

```

from nicegui import ui

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    space = ui.space().classes('bg-red-100')
    ui.label('右侧')

# 3 秒后删除空间，元素靠拢
ui.timer(3, space.delete)

ui.run()

```

(2) `move(target_container: Element, target_index: int, target_slot: str)`

将 `ui.space` 移动到其他弹性容器中：

```

from nicegui import ui

# 容器1：水平布局
with ui.row().classes('w-full border p-4 mb-2') as row1:
    ui.label('容器1-左')
    space = ui.space().classes('bg-blue-100')
    ui.label('容器1-右')

# 容器2：垂直布局
with ui.column().classes('h-32 border p-4') as col2:
    ui.label('容器2-上')
    ui.label('容器2-下')

# 3 秒后将空间从 row1 移动到 col2 的中间
ui.timer(3, lambda: space.move(target_container=col2, target_index=1))

ui.run()

```

(3) `ancestors()` / `descendants()`

遍历组件的祖先 / 后代元素（用于复杂布局树操作）：

```

from nicegui import ui

with ui.row() as parent:
    ui.label('左')
    space = ui.space()
    ui.label('右')

# 遍历空间的所有祖先元素（包含父容器 row）
for ancestor in space.ancestors(include_self=False):
    print(f'祖先元素: {ancestor}')

```

5. 资源与插槽

(1) `add_resource(path: str | Path)`

为 `ui.space` 添加外部资源（如自定义 CSS/JS 文件）：

```
# 引入外部 CSS 文件，定制空间样式
ui.space().add_resource('./custom-styles/space.css')
```

(2) `add_slot(name: str, template: str)`

添加 Vue 插槽（`ui.space` 默认无需插槽，仅在自定义扩展组件时使用）：

```
space = ui.space()
# 添加名为 "custom" 的插槽，插入自定义内容
space.add_slot('custom', '<span class="text-red-500">自定义填充内容</span>')
```

五、高级应用场景

1. 多元素加权分布

在弹性布局中使用多个 `ui.space`，通过 `flex-grow` 属性设置权重，实现元素的比例化分布：

```
from nicegui import ui

# 水平布局中，三个空间的权重比为 1:2:1
with ui.row().classes('w-full border p-4'):
    ui.label('元素1')
    ui.space(props='flex-grow=1 bg-blue-100')
    ui.label('元素2')
    ui.space(props='flex-grow=2 bg-green-100')
    ui.label('元素3')
    ui.space(props='flex-grow=1 bg-yellow-100')
    ui.label('元素4')

ui.run()
```

效果：元素 1 与元素 2 之间的空间宽度：元素 2 与元素 3 之间的空间宽度：元素 3 与元素 4 之间的空间宽度 = 1:2:1。

2. 表单布局对齐

利用 `ui.space` 实现表单标签与输入框的对齐，适配不同长度的标签文本：

```
from nicegui import ui

with ui.column().classes('w-80 border p-4 gap-3'):
    # 表单行1
    with ui.row().classes('items-center'):
        ui.label('用户名: ').classes('w-24') # 固定标签宽度
```



```

        ui.input().classes('flex-1') # 输入框占剩余空间
# 表单行2
with ui.row().classes('items-center'):
    ui.label('密码: ').classes('w-24')
    ui.input(password=True).classes('flex-1')
# 表单行3（带说明文本）
with ui.row().classes('items-center'):
    ui.label('备注: ').classes('w-24')
    ui.input().classes('flex-1')
    ui.space() # 填充输入框与说明文本间的空间
    ui.label('可选').classes('text-gray-500')

ui.run()

```

3. 响应式布局切换

结合 `ui.space` 的可见性绑定，实现不同屏幕尺寸下的布局切换：

```

from nicegui import ui

# 响应式逻辑：屏幕宽度 < 640px 时隐藏空间，元素靠拢
def update_layout(e):
    is_small_screen = e.args['width'] < 640
    space.set_visibility(not is_small_screen)

# 监听窗口尺寸变化
ui.window_size().on('update', update_layout)

with ui.row().classes('w-full border p-4'):
    ui.label('左侧')
    space = ui.space().classes('bg-blue-100')
    ui.label('右侧')

ui.run()

```

效果：大屏时元素两端对齐，小屏时元素靠拢，适配移动端显示。

4. 卡片内垂直布局

在 `ui.card` 中使用 `ui.space`，将操作按钮固定在卡片底部：

```

from nicegui import ui

with ui.card().classes('w-64 border p-4'):
    ui.label('卡片标题').classes('text-lg font-bold')
    ui.label('这是卡片内容，可能有多行文本...')
    ui.space() # 填充内容与按钮间的垂直空间
    ui.button('操作按钮').classes('w-full')

ui.run()

```

效果：无论卡片内容多少，按钮始终固定在底部。

六、注意事项

1. 布局依赖限制：`ui.space` 仅在弹性布局容器（`display: flex`）中生效，若父容器为非弹性布局（如 `block`），则无法填充空间；
2. 样式优先级：`style` 内联样式 > `classes` > `default_style` > 框架默认样式；
3. 空间收缩与增长：默认支持 `flex-grow: 1`（自动增长）和 `flex-shrink: 1`（自动收缩），可通过 `props` 手动设置 `flex-grow`/`flex-shrink` 调整行为；
4. 版本兼容性：`html_id` 属性需 NiceGUI 2.16.0+ 支持，`toggle` 类操作需 2.7.0+ 支持，`on` 方法同时支持 Python 和 JS 处理器需 2.18.0+ 支持；
5. 隐藏后的影响：`set_visibility(False)` 或删除 `ui.space` 后，原填充空间会消失，周围元素会自动靠拢，需注意布局联动效果。

总结

`ui.space` 是 NiceGUI 中用于弹性布局空间填充的核心组件，通过自动适配容器剩余空间，简化了“两端对齐”“垂直分布”“比例化布局”等复杂需求的实现。其优势在于无需手动计算尺寸，支持样式定制、事件绑定、动态控制等扩展能力，适用于表单、卡片、导航栏等各类界面布局场景。无论是简单的元素对齐，还是复杂的响应式布局，`ui.space` 都能显著提升开发效率，让布局逻辑更简洁、更易维护。